

**AICTE - EDUSKILLS VIRTUAL INTERNSHIP
PROJECT REPORT**

"Markdown to HTML Converter"

SUBMITTED BY:

Student Name: Aman Maurya
Program: B.Tech
Branch: B.Tech(Computer Science and Engineering)

SUBMITTED TO:

IILM University, Greater Noida

Internship Details:

AICTE Student ID: STU699855d2d9881771591122
APAAR ID: 511386729061
Internship Duration: 8 Weeks (Jun 2026 - Jul 2026)
Supported by: Eduskills Academy

Under the Guidance of

Project Manager
Eduskills

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr./Ms. Aman Maurya**, a student of **IILM University, Greater Noida**, has successfully completed **8-Week (Jun 2026 - Jul 2026)** AICTE – EduSkills Virtual Internship Program.

During the internship, he/she worked on the project titled "**Markdown to HTML Converter**". His/her performance during the internship was found to be satisfactory, and he/she demonstrated good technical and professional skills.

We wish him/her success in future endeavors.

Sagarabala Behera

Mentor, EduSkills Academy

Date: 30-07-2026

DECLARATION

I hereby declare that the work presented in this Online Virtual Internship Project Report is my original work carried out under the guidance of my industry mentor and faculty guide. This work has not been submitted earlier for the award of any degree or diploma.

Aman Maurya

(Signature)

Date: 30-07-2026

ACKNOWLEDGEMENT

I express my sincere gratitude to **AICTE & EduSkills** for providing me with the opportunity to undertake this online virtual internship. I would like to thank my industry mentor **Mr./Ms. Sagarabala Behera** for his/her valuable guidance, continuous support, and technical insights throughout the internship period.

I am also thankful to the Department of **B.Tech(Computer Science and Engineering)** for their encouragement and support.

ABSTRACT

This project focuses on the development of a Markdown to HTML converter. The primary goal is to build a robust parser that transforms standard Markdown syntax into well-formed HTML, handling elements such as headers, text emphasis, inline code snippets, and links. The converter is designed as a sequential processing pipeline, ensuring accurate and efficient conversion.

Table of Contents

1. Introduction	2
2. Organization Profile	3
3. Internship Overview	4
4. Problem Statement	5
5. Methodology	6
6. Tools & Technologies	7
7. Implementation	8
8. Learning Outcomes	11
9. Challenges & Solutions	12
10. Conclusion	13
11. Future Scope	14
12. References	15
13. Annexure	16

1. Introduction

Industry-oriented learning through internships plays a crucial role in bridging the gap between academic knowledge and real-world applications. This online virtual internship provided hands-on exposure to **Python Full Stack Development With Project** and its application in modern software solutions.

2. Organization Profile

Organization Name: EduSkills Academy

Domain: EdTech & Skill Development

Services: Virtual Internships

Vision: To empower students with industry-relevant skills.

3. Internship Overview

- Internship Mode: **Online / Virtual**
- Duration: **8 Weeks**
- Platform Used: **EduSkills LMS**
- Role: **Intern**

4. Problem Statement & Objectives

Problem Statement:

The Markdown to HTML Converter is a software utility designed to automate the translation of plain text formatted with Markdown syntax into equivalent HTML code. This project tackles the common need for a simple yet effective way to render Markdown content for web display or other HTML-based platforms. It systematically addresses key Markdown features, including the conversion of various header levels, the application of text emphasis (bold and italics), the proper handling of inline code segments, and the correct parsing of hyperlinks. The final implementation consolidates these individual parsing modules into a cohesive pipeline, ensuring that a given Markdown input is processed efficiently and accurately to produce valid, semantic HTML output, making it a foundational tool for content creation and web development.

Objectives:

- To design and implement a solution for the given problem statement.
- To utilize modern tools and algorithms.
- To optimize performance and code quality.

5. Project Description & Methodology

The development of the Markdown to HTML converter was approached through a systematic, step-by-step implementation of core Markdown features. Initially, the project focused on parsing and converting Markdown headers by identifying specific prefix patterns (e.g., '#', '##', '###'). Subsequently, text emphasis, such as bold and italic text, was handled using regular expressions to find and replace patterns like '**text**' and '*text*'. Inline code snippets were parsed by matching content enclosed in backticks. Markdown links, in the format '[text](url)', were then implemented using regular expressions with capture groups to extract both the link text and the URL. Finally, all individual parsing steps were integrated into a comprehensive processing pipeline, allowing for sequential application of these transformations to a given Markdown string, resulting in valid HTML output.

6. Tools & Technologies Used

Programming Language: Python

Libraries: `re` (for regular expression operations)

Concepts: String manipulation, Prefix identification, Regular Expressions (pattern matching and replacement, capture groups), Sequential processing pipelines.

7. Implementation & Results

Problem 1: Converting Markdown Headers

Description: The problem of converting Markdown headers involves accurately identifying lines starting with specific hash symbols ('#', '##', '###', etc.) followed by a space, and transforming them into their corresponding HTML header tags (e.g., '<h1>', '<h2>', '<h3>'). This requires a mechanism to detect these prefixes and replace them with the appropriate HTML element.

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

results = []
for line in lines:
    line = line.strip()
    if not line:
        result = {"html": ""}
    else:
        # FIX THIS LOGIC
        if line.startswith("### "):
            res = "<h3>" + line[4:].strip() + "</h3>"
        elif line.startswith("## "):
            res = "<h2>" + line[3:].strip() + "</h2>"
        elif line.startswith("# "):
            res = "<h1>" + line[2:].strip() + "</h1>"
        else:
            res = line

        result = {"html": res}

    results.append(result)

print(json.dumps(results, separators=(',', ':')))
```

Problem 2: Text Emphasis Conversion

Description: This task focuses on converting common Markdown text emphasis syntax, such as double asterisks for bold text ('**bold**') and single asterisks for italic text ('*italic*'), into their respective HTML equivalents ('' and ''). The solution involves using regular expressions to identify these patterns within the text and replace them with the correct HTML tags.

```
import sys
import json
import re

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

results = []
for line in lines:
    processed = line.strip()

    # YOUR CODE HERE
    processed = re.sub(r'\s*(.+?)\s*', r'<strong>\1</strong>', processed)
    processed = re.sub(r'\s*(.+?)\s*', r'<em>\1</em>', processed)

    result = {"html": processed}
    results.append(result)

print(json.dumps(results, separators=(',', ':')))
```

Problem 3: Parsing Inline Code Snippets

Description: The challenge here is to correctly identify and convert inline code snippets, typically denoted by single backticks (e.g., 'code'), into HTML's `<code>` tags. This requires a method, likely using regular expressions, to match content enclosed within backticks and perform the necessary HTML transformation.

```
import sys
import json
import re

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

results = []
for line in lines:
    processed = line.strip()

    # YOUR CODE HERE
    processed = re.sub(r'`(.*)`', r'<code>\1</code>', processed)

    result = {"html": processed}
    results.append(result)

print(json.dumps(results, separators=(',', ':')))
```

Problem 4: Converting Markdown Links

Description: This problem entails parsing Markdown links, which follow the format '[link text](url)', and converting them into HTML anchor tags ('<a>'). It requires extracting both the visible link text and the target URL using pattern matching, specifically leveraging regular expressions with capture groups to isolate these two components for constructing the '<a>' tag.

```
import sys
import json
import re

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

results = []
for line in lines:
    processed = line.strip()

    # YOUR CODE HERE
    processed = re.sub(r'\[(.*?)\]\((.*?)\)', r'<a href="\2">\1</a>', processed)

    result = {"html": processed}
    results.append(result)

print(json.dumps(results, separators=(',', ':')))
```

Problem 5: The Full Markdown Pipeline

Description: The final problem is to integrate all previously developed parsing functionalities into a single, coherent processing pipeline. This involves defining a sequence of operations where each Markdown feature is processed in turn, ensuring that the output of one stage serves as the input for the next, culminating in a complete conversion of the entire Markdown document to HTML.

```

import sys
import json
import re

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

results = []
for line in lines:
    processed = line.strip()

    # INTEGRATE ALL RULES
    processed = re.sub(r'\[(.*?)\]\((.*?)\)', r'<a href="\2">\1</a>', processed)
    processed = re.sub(r'\*(.*?)\*', r'<strong>\1</strong>', processed)
    processed = re.sub(r'\*(.*?)\*', r'<em>\1</em>', processed)
    processed = re.sub(r'\`(.*?)`', r'<code>\1</code>', processed)

    if processed.startswith("### "):
        processed = "<h3>" + processed[4:] + "</h3>"
    elif processed.startswith("## "):
        processed = "<h2>" + processed[3:] + "</h2>"
    elif processed.startswith("# "):
        processed = "<h1>" + processed[2:] + "</h1>"

    result = {"html": processed}
    results.append(result)

print(json.dumps(results, separators=(',', ':')))
```

8. Learning Outcomes

- Gained practical experience in parsing and transforming text using Python.
- Developed a deeper understanding of regular expressions for pattern matching and text manipulation.
- Learned how to design and implement a sequential processing pipeline for complex tasks.
- Improved problem-solving skills by breaking down a large task into smaller, manageable components.

9. Challenges Faced & Solutions

Challenge: Accurately handling nested or overlapping Markdown syntax, especially with regular expressions.

Solution: Carefully crafted regex patterns with non-greedy matching and prioritized order of operations in the pipeline to avoid unintended substitutions.

Challenge: Ensuring the generated HTML is valid and semantically correct for all supported Markdown features.

Solution: Rigorous testing with various Markdown inputs and comparing the output against expected HTML structures, refining the parsing logic as needed.

10. Conclusion

The Markdown to HTML Converter project successfully demonstrates the feasibility of creating a functional parser for a significant subset of Markdown syntax. By systematically addressing core features and integrating them into a processing pipeline, the project provides a valuable tool for content conversion and highlights the power of string manipulation and regular expressions in programming.

11. Future Scope

- Extend support to include additional Markdown features like lists (ordered and unordered), blockquotes, and images.
- Implement handling for more complex HTML structures and attributes.
- Develop a robust error-handling mechanism for malformed Markdown input.
- Consider performance optimizations for processing very large Markdown documents.

12. References

Python Documentation: <https://docs.python.org/3/>

Regular Expression HOWTO (Python Docs): <https://docs.python.org/3/howto/regex.html>

Markdown Syntax Guide: <https://www.markdownguide.org/basic-syntax/>

13. Annexure

- Internship Completion Certificate
- Offer Letter